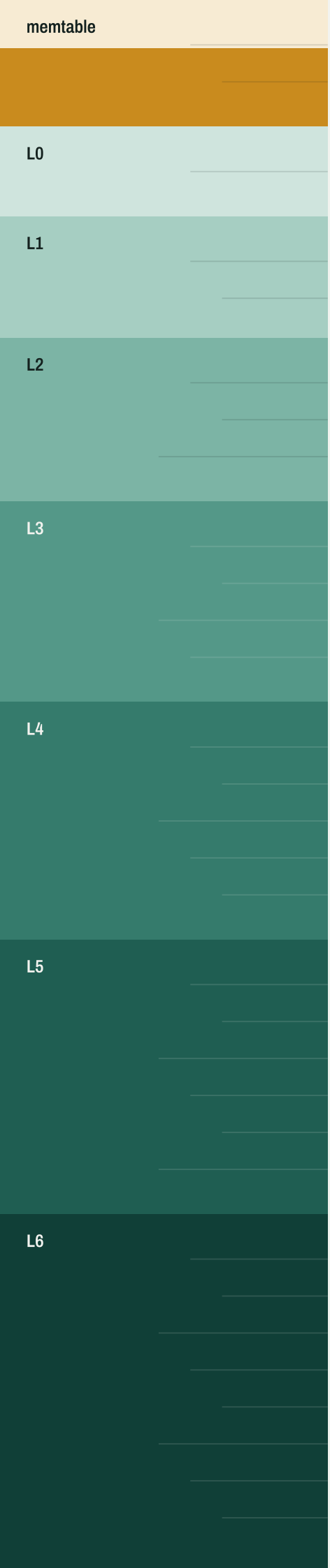




Banco de dados LSM-Tree

Arca

Manual técnico

Instalação, uso, operação, integração e extensão do banco de dados Arca. Servidor em C++20 com LSM-Tree, protocolo compatível com o Redis, tabelas com SQL, API REST em Go e painel web, do primeiro uso à operação em produção.



Memtable: escritas recentes, em memória

SSTables nos níveis L0 a L6, em disco, descendo pela compactação

Versão 1.2.0

Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 Visão geral 4

1.1 O que vem no projeto 4
1.2 Por que uma LSM-Tree 4
1.3 Garantias 4
1.4 Novidades da versão 1.2.0 5

02 Instalação 6

2.1 Com Docker (recomendado) 6
2.2 Compilando o servidor 6
2.3 Gateway e painel sem Docker 6

03 Primeiros passos 7

3.1 Entrando no painel 7
3.2 Usando o redis-cli 8
3.3 Usando o arca-cli 8

04 O servidor e os comandos 9

4.1 Como o servidor atende 9
4.2 Comandos 9
4.3 Comandos novos da 1.2.0 9
4.4 Expiração de chaves 10
4.5 Transações 10
4.6 Snapshot por conexão 11

05 SQL 12

5.1 Tipos 12
5.2 Instruções 12
5.3 Planos de execução 12
5.4 Regras de escrita 13
5.5 Consultas salvas e histórico 1.2.0 13

06 O painel 15

6.1 Telas 15
6.2 Paleta de comandos e atalhos 16
6.3 Barra de status 17
6.4 Identidade visual 17

07 Integrações 22

Copyright (©) 2026 Erik Pimenta

7.1 Conectando aplicações 22
7.2 Chaves de acesso 23
7.3 Webhooks 23
7.4 Prometheus 25

08 Exportar e importar 27

8.1 Chaves em JSON 27
8.2 Tabelas em CSV ou JSON 27
8.3 Boas práticas 28

09 API REST 29

9.1 Rotas 29
9.2 Exemplos 30

10 Configuração 31

10.1 Servidor 31
10.2 Gateway 32

11 Operação 33

11.1 Backup com checkpoints 33
11.2 Verificação de integridade 33
11.3 Monitoramento 33
11.4 Ajuste de desempenho 34
11.5 Medindo 35
11.6 Inspecionando os arquivos 35

12 Estendendo a Arca 36

12.1 Um comando novo 36
12.2 Uma função SQL 36
12.3 Embutindo o motor 36
12.4 Gateway e painel 36

13 Solução de problemas 37

13.1 Problemas comuns 37

Copyright (©) 2026 Erik Pimenta

01 Visão geral

A Arca é um banco de dados que guarda dois tipos de dado no mesmo motor de armazenamento: pares chave-valor, no estilo do Redis, e tabelas com colunas tipadas consultadas por SQL.

Tudo é gravado em arquivos binários próprios organizados como uma LSM-Tree (Log-Structured Merge-Tree), a mesma família de estrutura usada por LevelDB, RocksDB e Cassandra.

1.1 O que vem no projeto

O projeto é dividido em programas que podem rodar juntos ou separados:

Programa	Linguagem	Função
`arca-server`	C++20	Motor LSM, rede TCP assíncrona, comandos, SQL, expiração de chaves, checkpoints e verificação de integridade
`arca-gateway`	Go 1.24	API REST, sessões, chaves de acesso, papéis, auditoria, webhooks, métricas para o Prometheus, eventos em tempo real e entrega do painel
Painel (`web/`)	React + TypeScript	Interface para ver os estratos da LSM-Tree, navegar em chaves e tabelas, rodar SQL, importar e exportar dados e configurar integrações
`arca-cli` , `arca-bench` , `arca-inspect`	C++20	Console, teste de carga e inspeção dos arquivos binários

1.2 Por que uma LSM-Tree

Bancos baseados em B-Tree atualizam páginas no lugar, o que gera escrita aleatória no disco. A LSM-Tree transforma toda escrita em escrita sequencial: primeiro no log, depois em uma estrutura em memória e, quando ela enche, em um arquivo ordenado e imutável. Arquivos antigos são mesclados em segundo plano.

O resultado é alta vazão de escrita, recuperação simples após queda e leituras rápidas com a ajuda de filtros de Bloom e cache de blocos.

1.3 Garantias

- **Durabilidade:** com `appendfsync always` , uma escrita confirmada sobrevive a queda de energia. Com `everysec` (padrão), perde-se no máximo cerca de um segundo.
- **Atomicidade:** cada comando e cada instrução SQL viram um único lote gravado de uma vez no log.
- **Isolamento:** leituras usam um número de sequência e nunca enxergam lotes pela metade. `MULTI` / `EXEC` executa os comandos enfileirados sem intercalar outros clientes.
- **Integridade:** todo registro do log e todo bloco de SSTable têm checksum CRC32C, e `ARCA.VERIFY` confere todos eles com o servidor no ar.

Copyright (©) 2026 Erik Pimenta

1.4 Novidades da versão 1.2.0

A versão 1.2.0 traz um visual próprio para o painel, ferramentas de operação e várias formas de integrar a Arca a outros sistemas. O formato dos arquivos em disco não muda: uma pasta criada pela 1.1.0 abre na 1.2.0 sem conversão.

Área	O que mudou
Visual do painel	Identidade "estratos e pátina": a coluna de estratos da visão geral mostra a memtable e os níveis L0 a L6 como camadas, com a memória em âmbar e o disco em verde-pátina. Tipografia Archivo e IBM Plex Mono
Navegação	Paleta de comandos (`Ctrl K`), atalhos de teclado (`g v` , `g c` , `g t` ...) e barra de status fixa com conexão, porta, fsync, operações por segundo e uso da memtable
Integrações	Tela nova com exemplos de conexão em várias linguagens, chaves de acesso (tokens de API com papel e validade), webhooks assinados com HMAC-SHA256 e o endpoint `/metrics` para o Prometheus
Dados	Exportar e importar chaves (JSON) e tabelas (CSV ou JSON); consultas salvas e compartilhadas no Console SQL, com exportação do resultado em CSV
Operação	Lista e remoção de checkpoints (`ARCA.CHECKPOINTS`) e verificação de integridade das SSTables com o servidor no ar (`ARCA.VERIFY`)
Comandos	`GETEX` , `GETRANGE` , `SETRANGE` , `COPY` , `TOUCH` , `RANDOMKEY` , `EXPIRETIME` , `PEXPIRETIME` , `MSETNX` e `MEMORY USAGE`

Copyright (©) 2026 Erik Pimenta

02 Instalação

O caminho mais simples é o Docker, que compila e sobe o servidor, o gateway e o painel juntos. Também dá para compilar tudo sem Docker.

2.1 Com Docker (recomendado)

É o caminho mais simples em qualquer sistema. Instale o Docker Desktop (Windows e macOS) ou o Docker Engine com Compose (Linux).

1. Baixe o repositório e abra a pasta `Persistent-Key-Value-Server`.
2. Windows: dê dois cliques em `iniciar.cmd`. Linux ou macOS: rode `./iniciar.sh`.
3. Na primeira vez o script cria o arquivo `.env` com uma senha aleatória para o servidor, compila as três imagens e sobe os contêineres.
4. Quando o painel responder, o navegador abre em `http://localhost:8080`.

```
cp .env.example .env                # sem o script: troque ARCA_PASSWORD
docker compose up -d --build
docker compose logs -f arca         # acompanha o servidor
docker compose down                 # para tudo (os dados ficam no volume "dados")
```

2.2 Compilando o servidor

O núcleo usa só a biblioteca padrão do C++. Requisitos: GCC 11+, Clang 14+ ou Visual Studio 2022, e CMake 3.20+.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
./build/arca-tests                  # testes do motor, do SQL e do servidor
./build/arca-server config/arca.conf
```

No Windows, use `cmake --build build --config Release`; os executáveis ficam em `build\Release\`. A rede usa epoll no Linux, poll no macOS e WSAPoll no Windows, escolhidos na compilação.

2.3 Gateway e painel sem Docker

```
cd web && npm ci && npm run build        # gera web/dist
cd ../gateway
ARCA_ADDR=127.0.0.1:7379 ARCA_PASSWORD=senha go run ./cmd/arca-gateway
```

O gateway procura o painel em `../web/dist`; mude com `ARCA_WEB_DIR`. Para desenvolver o painel com recarga automática, rode `npm run dev` e abra `http://localhost:5173` (as chamadas para `/api` e `/metrics` vão para a porta 8080).

Copyright (©) 2026 Erik Pimenta

03 Primeiros passos

Com o servidor no ar, entre no painel com uma das contas de demonstração ou fale com o servidor direto pelo `redis-cli` ou pelo `arca-cli`.

3.1 Entrando no painel

Na primeira subida o gateway cria três contas de demonstração e dados de exemplo. Todas usam a senha `Arca@2026`.

Conta	Papel	Permissões
`admin@arca.dev`	Administrador	Tudo, inclusive usuários, configuração, auditoria, webhooks e console de comandos
`operador@arca.dev`	Operador	Lê e grava chaves, linhas e SQL; importa dados; grava a memtable, compacta, cria checkpoints e verifica a integridade
`leitura@arca.dev`	Leitura	Só consulta e exporta

As contas existem só para teste. Em uso real, suba com `ARCA_SEED=false` num volume vazio, crie um administrador e troque as senhas.

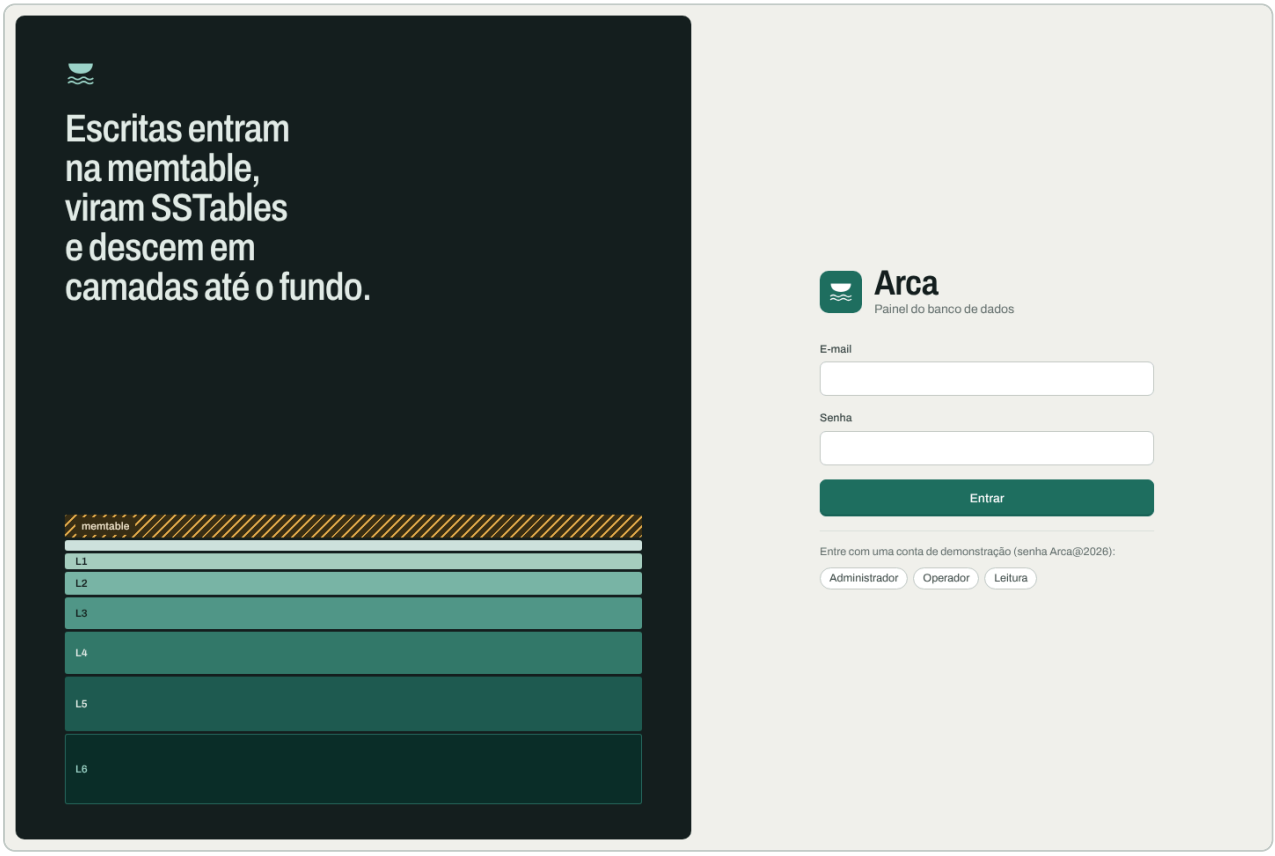


Figura 3.1 Tela de entrada do painel com as contas de demonstração

Copyright (©) 2026 Erik Pimenta

3.2 Usando o redis-cli

```
redis-cli -p 7379 -a "$ARCA_PASSWORD"
127.0.0.1:7379> SET sessao:42 ativo EX 3600
OK
127.0.0.1:7379> TTL sessao:42
(integer) 3600
127.0.0.1:7379> INCRBY contador:visitas 5
(integer) 5
127.0.0.1:7379> GETEX sessao:42 EX 7200
"ativo"
```

3.3 Usando o arca-cli

O `arca-cli` abre um console que aceita comandos e SQL. Linhas que começam com `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE`, `DROP`, `ALTER`, `SHOW`, `DESCRIBE` ou `EXPLAIN` são enviadas como SQL e o resultado aparece em tabela.

```
$ arca-cli -a senha
127.0.0.1:7379> SELECT categoria, COUNT(*) AS itens FROM produtos GROUP BY categoria
+-------------+-------+
| categoria   | itens |
+-------------+-------+
| Papelaria   | 9     |
| Eletrônicos | 8     |
+-------------+-------+
(2 linhas)
```

Copyright (©) 2026 Erik Pimenta

04 O servidor e os comandos

O `arca-server` fala RESP2, o protocolo do Redis, então `redis-cli` e as bibliotecas de Redis funcionam sem mudança.

4.1 Como o servidor atende

O `arca-server` abre um socket de escuta e cria `io-threads` laços de eventos. O primeiro laço aceita conexões e as distribui em rodízio. Cada laço lê os dados dos clientes sem bloquear, decodifica o protocolo RESP2, executa o comando e devolve a resposta. Vários comandos podem chegar de uma vez (pipelining) e são respondidos na ordem.

Comandos que podem demorar (SQL, `KEYS`, `DBSIZE`, `FLUSHDB`, compactação, checkpoint, `ARCA.VERIFY`) vão para um pool de `worker-threads`. Enquanto isso a conexão fica pausada para preservar a ordem das respostas, mas o laço continua atendendo os outros clientes.

4.2 Comandos

Grupo	Comandos
Conexão	`PING` · `ECHO` · `AUTH` · `HELLO` · `SELECT 0` · `QUIT` · `CLIENT ID/SETNAME/GETNAME/LIST`
Strings	`GET` · `SET` (`EX PX EXAT PXAT NX XX GET KEEPTTL`) · `SETNX` · `SETEX` · `PSETEX` · `GETSET` · `GETDEL` · `GETEX` · `GETRANGE` · `SUBSTR` · `SETRANGE` · `MGET` · `MSET` · `MSETNX` · `APPEND` · `STRLEN`
Contadores	`INCR` · `DECR` · `INCRBY` · `DECRBY` · `INCRBYFLOAT`
Chaves	`DEL` · `UNLINK` · `EXISTS` · `TOUCH` · `TYPE` · `COPY` · `RENAME` · `RENAMENX` · `RANDOMKEY` · `EXPIRE` · `PEXPIRE` · `EXPIREAT` · `PEXPIREAT` · `EXPIRETIME` · `PEXPIRETIME` · `TTL` · `PTTL` · `PERSIST` · `SCAN` · `KEYS` · `DBSIZE` · `FLUSHDB` · `MEMORY USAGE`
Transações	`MULTI` · `EXEC` · `DISCARD`
SQL	`SQL consulta` · `SQL.JSON consulta` · `SQL.TABLES`
Servidor	`INFO [seção]` · `CONFIG GET/SET/RESETSTAT` · `SLOWLOG GET/LEN/RESET` · `SAVE` · `BGSAVE` · `LASTSAVE` · `TIME` · `SHUTDOWN [NOSAVE]`
Arca	`ARCA.STATS` · `ARCA.FLUSH` · `ARCA.COMPACT` · `ARCA.CHECKPOINT [nome]` · `ARCA.CHECKPOINTS [DELETE nome]` · `ARCA.VERIFY` · `ARCA.SNAPSHOT BEGIN/END` · `ARCA.SCANJSON cursor padrão quantidade`

4.3 Comandos novos da 1.2.0

Os comandos novos seguem o comportamento do Redis 7, para que bibliotecas existentes funcionem sem adaptação.

Copyright (©) 2026 Erik Pimenta

Sintaxe	O que faz
`GETEX chave [EX s | PX ms | EXAT ts | PXAT ms | PERSIST]`	Lê o valor e, na mesma operação, define ou remove o TTL
`GETRANGE chave início fim`	Trecho do valor; índices negativos contam do fim. `SUBSTR` é sinônimo
`SETRANGE chave posição valor`	Sobrescreve a partir da posição, completando com zeros se preciso; devolve o novo tamanho
`COPY origem destino [REPLACE]`	Copia o valor e o TTL; sem `REPLACE`, não sobrescreve um destino que já existe
`TOUCH chave [chave …]`	Conta quantas das chaves existem
`RANDOMKEY`	Uma chave qualquer do espaço chave-valor, ou nulo se estiver vazio
`EXPIRETIME chave` · `PEXPIRETIME chave`	Instante absoluto da expiração (segundos ou milissegundos); −1 sem TTL, −2 se a chave não existe
`MSETNX chave valor [chave valor …]`	Grava todas só se nenhuma existir; 1 se gravou, 0 se não
`MEMORY USAGE chave`	Bytes aproximados ocupados pela chave: chave, valor e controle
`ARCA.CHECKPOINTS`	Lista os checkpoints em JSON: `[{"name","created_ms","bytes","files"}]`
`ARCA.CHECKPOINTS DELETE nome`	Apaga um checkpoint (o nome só aceita letras, números, `.`, `-` e `_`)
`ARCA.VERIFY`	Confere o CRC32C de todos os blocos de todas as SSTables vivas e devolve um relatório em JSON (capítulo 11)

```
127.0.0.1:7379> SET nota "Arca 1.2"
OK
127.0.0.1:7379> SETRANGE nota 5 "1.3"
(integer) 8
127.0.0.1:7379> COPY nota nota:copia
(integer) 1
127.0.0.1:7379> MEMORY USAGE nota
(integer) 46
```

4.4 Expiração de chaves

Uma chave com TTL guarda o instante de expiração junto do valor e ganha uma entrada num índice ordenado por tempo. Leituras já tratam a chave vencida como inexistente; uma thread percorre esse índice a cada 100 ms e apaga as chaves vencidas de fato, liberando espaço na próxima compactação.

4.5 Transações

Depois de `MULTI`, os comandos são validados e enfileirados. `EXEC` roda todos em sequência com exclusividade: nenhum outro cliente executa nada no meio. Se algum comando enfileirado tiver erro de sintaxe, `EXEC` responde `EXECABORT` e nada é aplicado.

Copyright (©) 2026 Erik Pimenta

4.6 Snapshot por conexão

`ARCA.SNAPSHOT BEGIN` fixa um retrato do banco para a conexão: até `ARCA.SNAPSHOT END`, `GET`, `SCAN` e `SELECT` enxergam os dados como estavam naquele instante, mesmo com escritas e compactações acontecendo.

Copyright (©) 2026 Erik Pimenta

05 SQL

As tabelas vivem na mesma LSM-Tree das chaves. Cada linha é um par chave-valor cuja chave é o identificador da tabela mais a chave primária, codificada de forma que a ordem dos bytes respeite a ordem dos valores.

5.1 Tipos

Isso permite busca direta pela chave primária e varreduras ordenadas. Os tipos aceitos são:

Tipo	Aceita	Observação
`INT`	INT, INTEGER, BIGINT, SMALLINT, TIMESTAMP	Inteiro de 64 bits
`REAL`	REAL, FLOAT, DOUBLE, NUMERIC, DECIMAL	Ponto flutuante de 64 bits; aceita literal inteiro
`TEXT`	TEXT, VARCHAR(n), STRING, CHAR	UTF-8, sem limite de tamanho
`BOOL`	BOOL, BOOLEAN	TRUE ou FALSE

5.2 Instruções

```
CREATE TABLE [IF NOT EXISTS] t (id INT PRIMARY KEY AUTOINCREMENT, nome TEXT NOT NULL, ativo
BOOL DEFAULT TRUE)
DROP TABLE [IF EXISTS] t
ALTER TABLE t ADD COLUMN email TEXT [DEFAULT 'x']
CREATE INDEX [IF NOT EXISTS] idx ON t (coluna)
DROP INDEX [IF EXISTS] idx
INSERT INTO t [(colunas)] VALUES (...), (...)
SELECT * | expressão [AS apelido], COUNT(*), SUM(c), AVG(c), MIN(c), MAX(c) FROM t
   [WHERE expr] [GROUP BY c, ...] [ORDER BY c [ASC|DESC], ...] [LIMIT n [OFFSET m]]
UPDATE t SET c = expr, ... [WHERE expr]
DELETE FROM t [WHERE expr]
EXPLAIN SELECT ...
SHOW TABLES
DESCRIBE t
```

Expressões aceitam `= != <> < <= > >=`, `AND OR NOT`, `IS [NOT] NULL`, `[NOT] LIKE` (com `%` e `_`, sem diferenciar maiúsculas), `[NOT] IN (...)`, `[NOT] BETWEEN a AND b`, aritmética `+ - * / %`, concatenação `||` e as funções `LOWER`, `UPPER`, `LENGTH`, `ABS`, `ROUND` e `COALESCE`.

5.3 Planos de execução

Plano	Quando é escolhido	Custo
busca pela chave primária	`WHERE` tem `pk = literal`	Uma leitura pontual

Copyright (©) 2026 Erik Pimenta

Plano	Quando é escolhido	Custo
busca pelo índice	`WHERE` tem `coluna_indexada = literal`	Uma varredura curta no índice e uma leitura por linha
varredura completa	Demais casos	Lê todas as linhas da tabela

```
EXPLAIN SELECT * FROM pedidos WHERE status = 'pago'
plano: busca pelo índice idx_pedidos_status em pedidos
```

5.4 Regras de escrita

- Toda tabela precisa de chave primária. Colunas `AUTOINCREMENT` recebem o próximo número quando o valor é omitido ou `NULL`.
- Chave primária duplicada e `NULL` em coluna `NOT NULL` rejeitam a instrução inteira; nada é gravado.
- Cada instrução vira um único lote: linhas, entradas de índice e contadores mudam juntos.
- Escritas numa tabela são serializadas por uma trava de leitura e escrita da própria tabela; tabelas diferentes não se bloqueiam.

5.5 Consultas salvas e histórico 1.2.0

O Console SQL guarda as últimas consultas executadas neste navegador e permite **salvar** uma consulta com nome no servidor. Uma consulta salva pode ser só sua ou **compartilhada** com todas as contas do painel; quem a criou (ou um administrador) pode apagá-la. O resultado de qualquer consulta pode ser baixado em CSV.

- `Ctrl Enter` executa; `Ctrl S` salva a consulta do editor.
- Pela API: `GET /api/queries`, `POST /api/queries {name, query, shared}` e `DELETE /api/queries/{id}`.

Copyright (©) 2026 Erik Pimenta

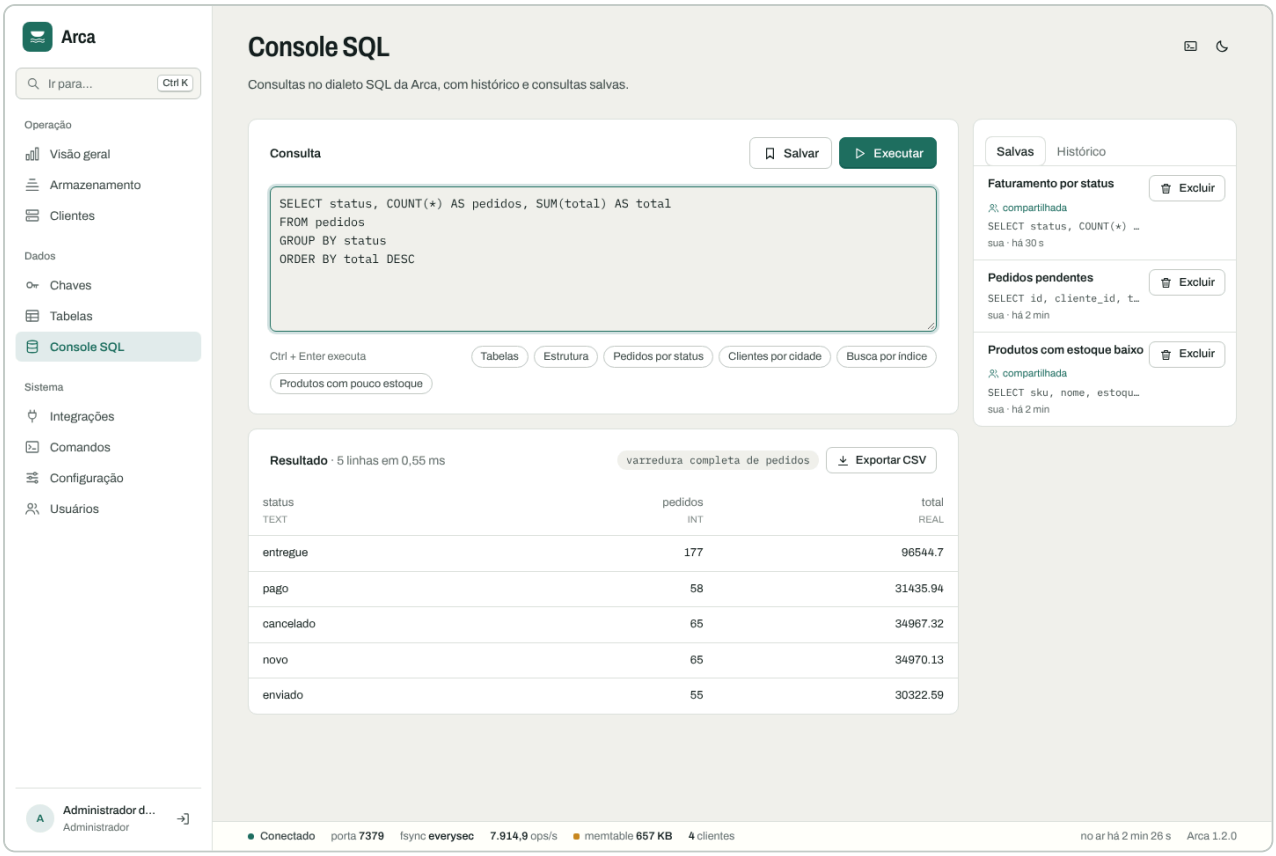


Figura 5.1 Console SQL: editor, consultas salvas, resultado em grade com o plano escolhido e o histórico

Copyright (©) 2026 Erik Pimenta

06 O painel

O painel mostra o estado do motor em tempo real (a cada segundo, por Server-Sent Events) e permite operar o banco sem linha de comando. Ele se adapta ao celular e tem tema claro e escuro.

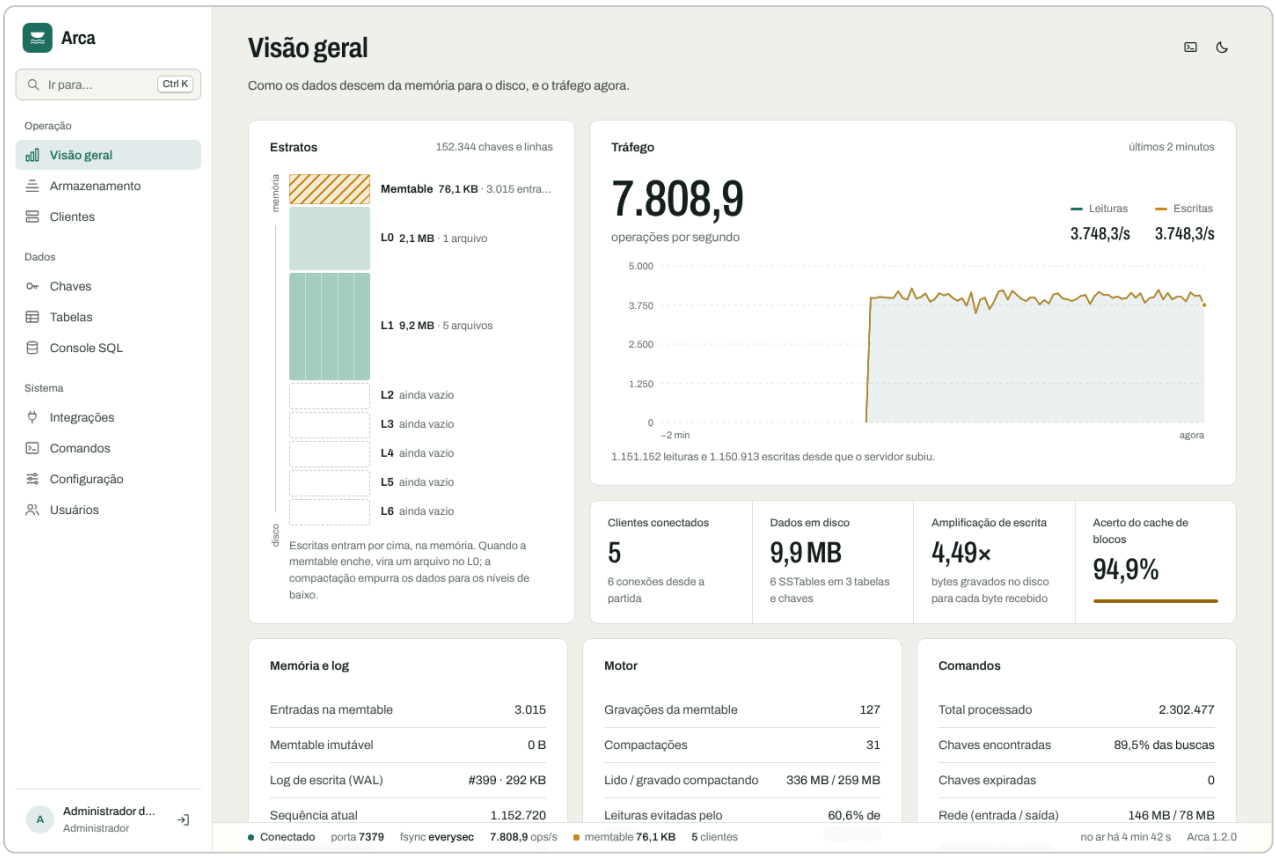


Figura 6.1 Visão geral: a coluna de estratos com a memtable e os níveis L0 a L6, as operações por segundo e os indicadores do motor

6.1 Telas

Tela	O que faz
Visão geral	Coluna de estratos (memtable enchendo e níveis com espessura proporcional ao tamanho), operações por segundo com o gráfico de leituras e escritas, clientes, chaves, cache de blocos e amplificação de escrita
Chaves	Busca por padrão (`sessao:*`), paginação por cursor, ver, editar, apagar e mudar o TTL; exportar e importar em JSON
Tabelas	Lista de tabelas com colunas e índices; grade com ordenação, filtro em SQL, paginação e edição de linhas; exportar em CSV ou JSON e importar CSV

Copyright (©) 2026 Erik Pimenta

Tela	O que faz
Console SQL	Editor com exemplos, histórico, consultas salvas e compartilhadas, resultado em grade, plano escolhido e download em CSV
Armazenamento	Memtable, WAL e cada SSTable por nível, compactações recentes, checkpoints (criar, listar, apagar) e verificação de integridade
Clientes	Conexões abertas e comandos lentos (slowlog)
Integrações	Exemplos de conexão por linguagem, chaves de acesso, webhooks e Prometheus
Comandos	Console direto do protocolo (só administrador)
Configuração	Parâmetros do servidor; os que podem mudar em execução são editáveis
Usuários	Contas, papéis, desbloqueio e a trilha de auditoria
Minha conta	Dados, permissões, troca de senha e tema

6.2 Paleta de comandos e atalhos

A paleta de comandos abre com `Ctrl K` (`⌘ K` no macOS) ou pelo botão **Ir para…** no cabeçalho. Digite parte do nome de uma tela ou ação e confirme com `Enter`: dá para ir a qualquer página, gravar a memtable, compactar, criar um checkpoint, abrir uma chave pelo nome, trocar o tema ou sair. A lista respeita o papel da conta: ações de operador não aparecem para quem só lê.

Atalho	Ação
`Ctrl K`	Abre a paleta de comandos
`g v`	Visão geral
`g c`	Chaves
`g t`	Tabelas
`g s`	Console SQL
`g a`	Armazenamento
`g l`	Clientes
`g i`	Integrações
`?`	Mostra a lista de atalhos
`Esc`	Fecha a paleta, o painel lateral ou o diálogo aberto

Os atalhos de duas teclas funcionam fora de campos de texto: aperte `g` e, em seguida, a letra da tela.

Copyright (©) 2026 Erik Pimenta

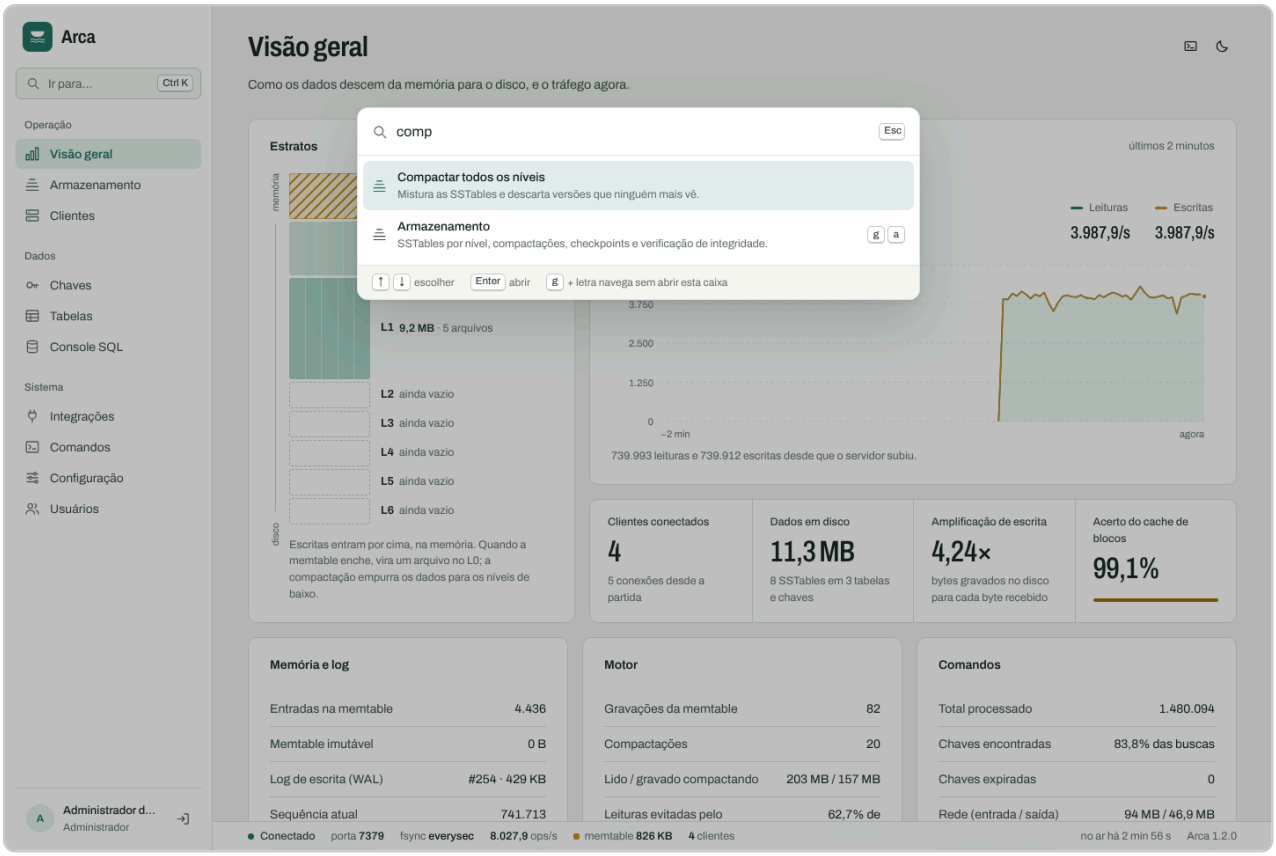


Figura 6.2 Paleta de comandos aberta com `Ctrl K`

6.3 Barra de status

No rodapé da área principal fica uma barra de status sempre visível, no estilo de um terminal. Ela mostra, da esquerda para a direita: o estado da conexão em tempo real (ao vivo, compactando, desconectado ou fora do ar), a porta do servidor, o modo de fsync, as operações por segundo, o uso da memtable em relação a `write-buffer-size` e a versão da Arca. Assim dá para acompanhar a saúde do banco em qualquer tela.

6.4 Identidade visual

A 1.2.0 troca o visual neutro por uma identidade própria, "estratos e pátina". A ideia vem do próprio motor: os dados descem em camadas, da memtable até o L6, como estratos de rocha. O que está em memória é quente (âmbar); o que já está em disco é frio, no verde-pátina do cobre oxidado de um baú antigo. A coluna de estratos da visão geral é o único elemento chamativo; o resto é papel frio, tinta escura e fios finos.

Elemento	Como aparece
Cores	Papel `#F2F3EF`, superfície `#FFFFFF`, tinta `#14211E`, pátina `#1E6F62` nos destaques e no foco, âmbar `#C98B1E` só para memória
Níveis da LSM-Tree	Do verde mais claro (L0, `#CFE4DD`) ao mais profundo (L6, `#103F37`), na coluna de estratos e no mapa de SSTables
Interface e números	Archivo variável: títulos com largura 82, números grandes condensados (largura 72) e algarismos tabulares
Código, chaves e SQL	IBM Plex Mono, só no que é código de verdade

Copyright (©) 2026 Erik Pimenta

Elemento	Como aparece
Forma	Raio de 10 px nos painéis e 7 px nos controles, sem sombras nos painéis, anel de foco de 2 px na cor pátina
Movimento	Só a memtable enchendo e a hachura da compactação descendo pela coluna; desligado quando o sistema pede menos movimento
Tema escuro	Os mesmos materiais em tons escuros, com a escolha guardada no navegador

As fontes são locais (pacotes Fontsource), sem CDN. Os tokens de cor e de tipografia ficam no topo de `web/src/styles.css`.

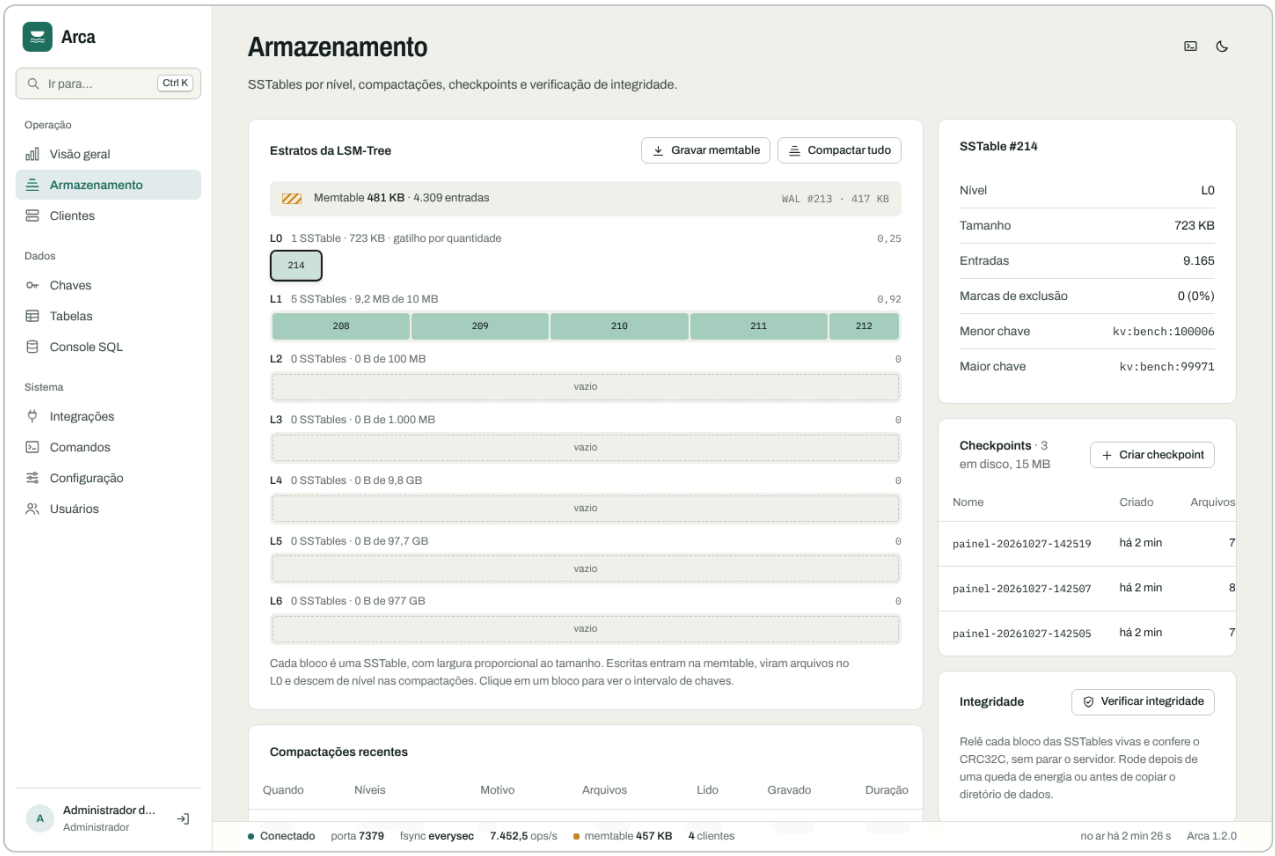


Figura 6.3 Armazenamento: cada bloco é uma SSTable, do L0 claro ao L6 profundo, com checkpoints e verificação de integridade

Copyright (©) 2026 Erik Pimenta

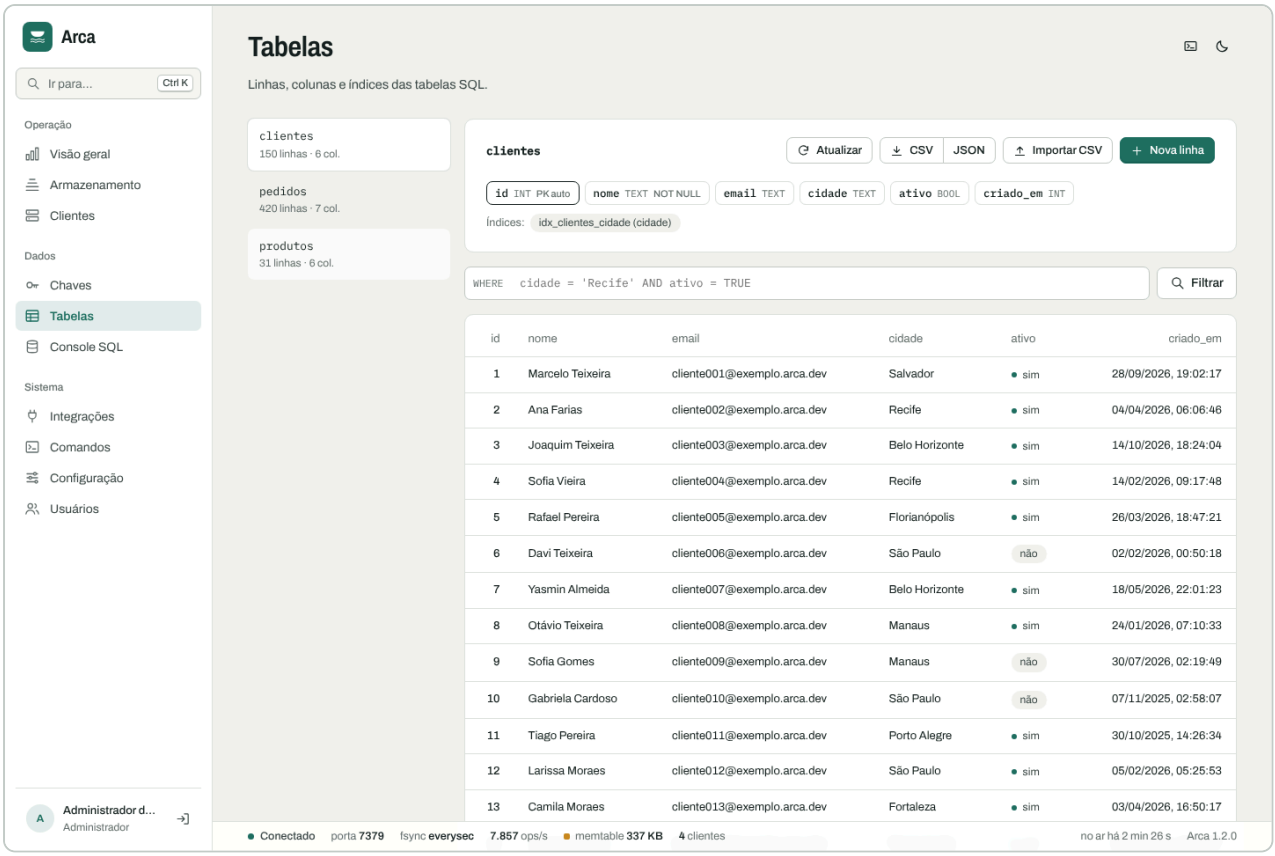


Figura 6.4 Tabelas: grade de linhas com filtro, ordenação, edição, exportação e importação

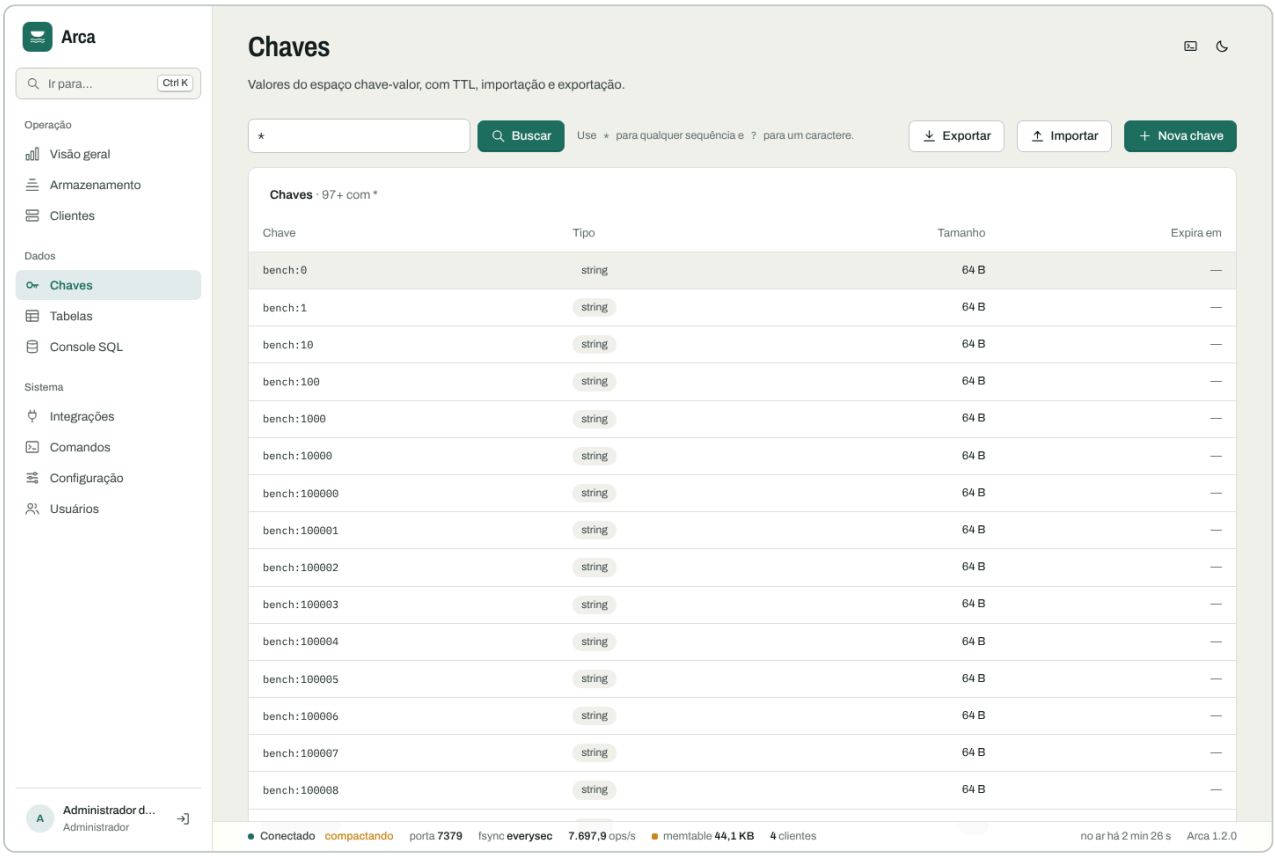


Figura 6.5 Chaves com TTL e tamanho, exportação e importação em JSON

Copyright (©) 2026 Erik Pimenta

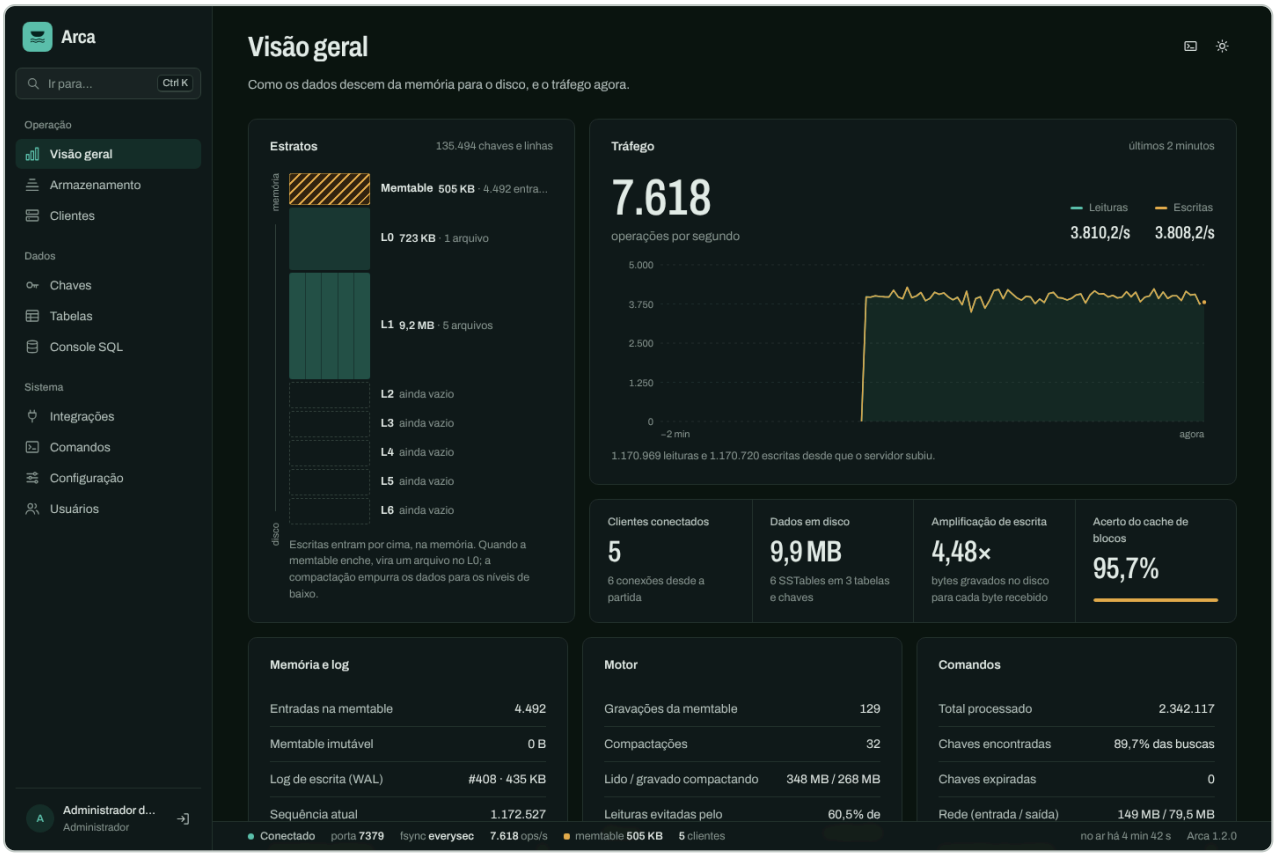


Figura 6.6 Painel no tema escuro

Copyright (©) 2026 Erik Pimenta

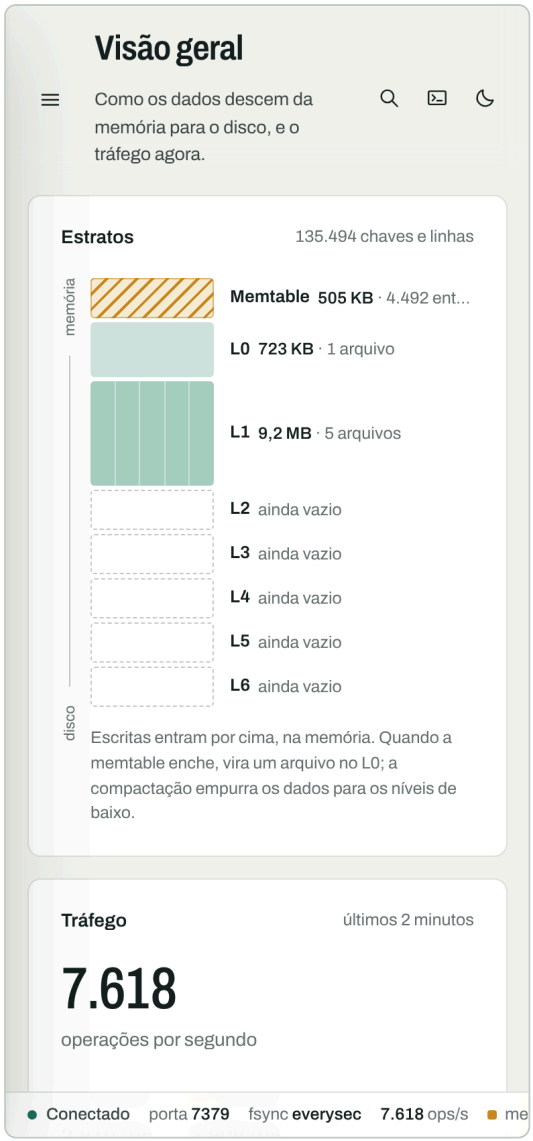


Figura 6.7 Painel no celular

Copyright (©) 2026 Erik Pimenta

07 Integrações

A Arca conversa com outros sistemas de quatro formas: pelo protocolo do Redis, pela API REST com chaves de acesso, por webhooks que avisam quando algo muda e pelo endpoint de métricas do Prometheus.

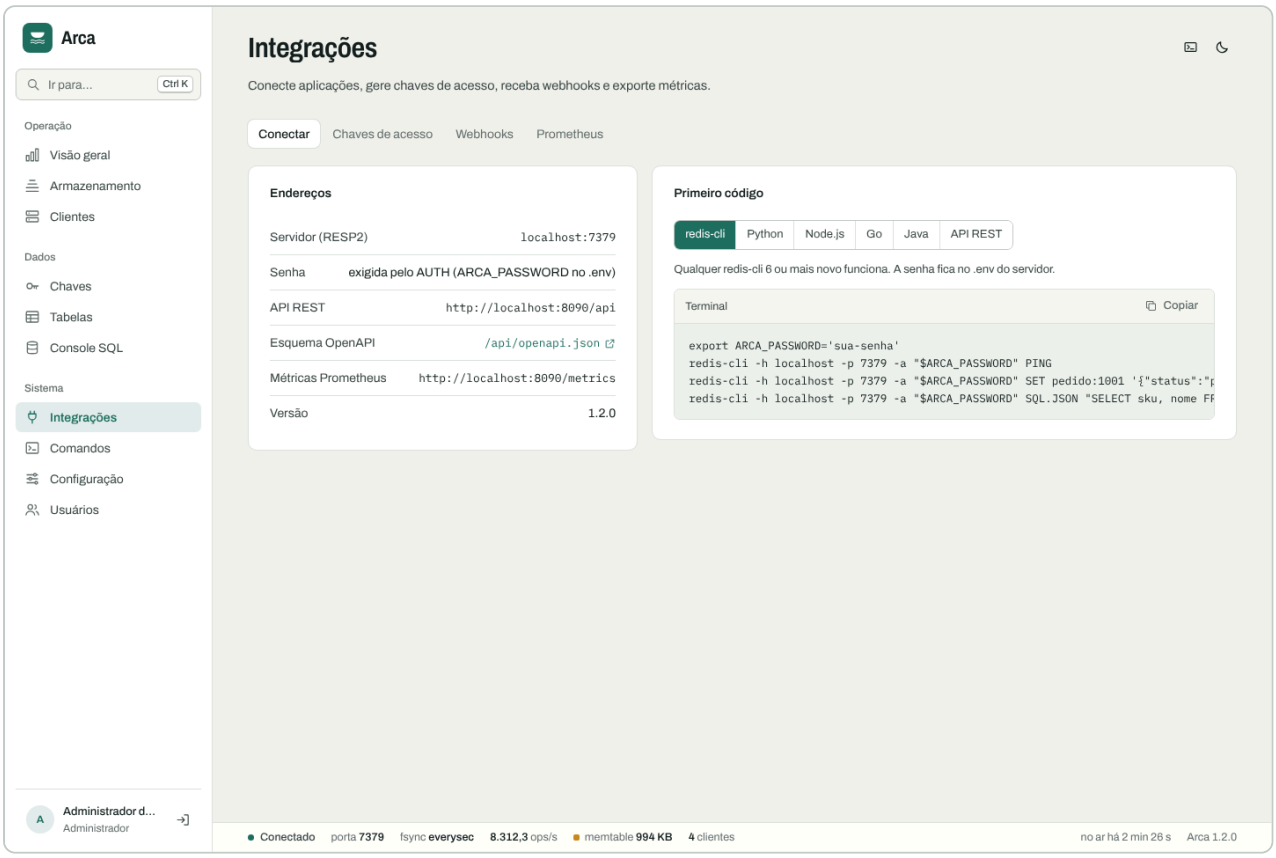


Figura 7.1 Integrações: exemplos de conexão, chaves de acesso, webhooks e Prometheus

7.1 Conectando aplicações

A tela **Integrações** mostra o endereço do servidor e exemplos prontos para copiar, já com o host e a porta corretos. Qualquer cliente de Redis funciona; a senha é a do `.env` (`ARCA_PASSWORD`).

Python (redis-py)

```
import redis
r = redis.Redis(host="localhost", port=7379, password="SENHA", decode_responses=True)
r.set("sessao:42", "ativo", ex=3600)
print(r.execute_command("SQL.JSON", "SELECT nome, preco FROM produtos LIMIT 3"))
```

Copyright (©) 2026 Erik Pimenta

Node.js (ioredis)

```
import Redis from "ioredis"
const r = new Redis({ host: "localhost", port: 7379, password: "SENHA" })
await r.set("sessao:42", "ativo", "EX", 3600)
const linhas = JSON.parse(await r.call("SQL.JSON", "SELECT * FROM clientes LIMIT 5"))
```

Go (go-redis)

```
rdb := redis.NewClient(&redis.Options{Addr: "localhost:7379", Password: "SENHA"})
rdb.Set(ctx, "sessao:42", "ativo", time.Hour)
json, _ := rdb.Do(ctx, "SQL.JSON", "SELECT COUNT(*) FROM pedidos").Text()
```

Java (Jedis)

```
try (Jedis j = new Jedis("localhost", 7379)) {
    j.auth("SENHA");
    j.setex("sessao:42", 3600, "ativo");
    Object json = j.sendCommand(() -> SafeEncoder.encode("SQL.JSON"), "SHOW TABLES");
}
```

7.2 Chaves de acesso

Para scripts, integrações e o Prometheus, use uma **chave de acesso** em vez da senha de uma pessoa. Cada chave tem nome, papel (Leitura, Operador ou Administrador, nunca acima do papel de quem a cria) e validade opcional em dias.

1. Abra **Integrações** → **Chaves de acesso** e clique em **Criar chave**.
2. Dê um nome que diga onde ela será usada (`prometheus-producao`, `etl-noturno`), escolha o papel e a validade.
3. Copie a chave exibida, que começa com `arca_`. Ela aparece **uma única vez**: o gateway guarda só o SHA-256.
4. Envie no cabeçalho `Authorization: Bearer arca_…`, em qualquer rota da API e em `/metrics`.

A lista mostra o prefixo, o papel, a criação, o último uso e a validade de cada chave; revogar apaga a chave na hora. Cada pessoa vê as suas chaves; um administrador vê todas com `?all=1`. A auditoria registra o e-mail do dono seguido do nome da chave.

```
curl -s localhost:8080/api/overview -H "Authorization: Bearer arca_3f9c…"

curl -s -X POST localhost:8080/api/tokens -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"name":"etl-
noturno","role":"operator","expires_days":90}'
```

7.3 Webhooks

Um webhook avisa outro sistema quando algo acontece na Arca: o gateway faz um `POST` com JSON para a URL cadastrada. Só administradores configuram webhooks, em **Integrações** → **Webhooks**.

Copyright (©) 2026 Erik Pimenta

Evento	Quando dispara
`key.write`	Uma chave foi gravada, importada ou teve o TTL alterado pelo painel ou pela API
`key.delete`	Uma chave foi apagada
`table.write`	Linhas inseridas, alteradas, apagadas ou importadas pela tela Tabelas
`sql.write`	Uma instrução SQL de escrita foi executada pelo Console SQL ou pela API
`storage.flush` · `storage.compact` · `storage.checkpoint`	Operações do motor
`config.change`	Um parâmetro do servidor foi alterado
`user.change`	Conta criada, alterada, desbloqueada ou removida
`auth.login`	Entrada no painel

Ao criar um webhook o painel mostra o **segredo** (`whsec_…`) uma única vez. Cada entrega leva os cabeçalhos `X-Arca-Event` , `X-Arca-Delivery` (identificador único) e `X-Arca-Signature: sha256=<hex>` , o HMAC-SHA256 do corpo com o segredo. O envio é assíncrono, com tempo limite de 5 segundos e até 2 tentativas; o painel mostra as últimas 30 entregas, o status HTTP e o tempo de cada uma, e o botão **Enviar teste** dispara uma entrega na hora.

Corpo de uma entrega

```
{"id":"d7c1…","event":"key.write","at":1793110156760,"actor":"operador@arca.dev","detail":"sessao:42"}
```

Confira a assinatura antes de confiar no conteúdo, sempre sobre o corpo exatamente como chegou e com comparação em tempo constante:

Python (Flask)

```
import hmac, hashlib
from flask import Flask, request, abort

SEGREDO = b"whsec_..."
app = Flask(__name__)

@app.post("/arca")
def arca():
    corpo = request.get_data()
    esperado = "sha256=" + hmac.new(SEGREDO, corpo, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(esperado, request.headers.get("X-Arca-Signature", "")):
        abort(401)
    evento = request.get_json()
    print(evento["event"], evento["detail"])
    return "", 204
```

Copyright (©) 2026 Erik Pimenta

Node.js (Express)

```
import crypto from "node:crypto"
import express from "express"

const SEGREDO = "whsec_..."
const app = express()

app.post("/arca", express.raw({ type: "application/json" }), (req, res) => {
  const esperado = "sha256=" + crypto.createHmac("sha256",
SEGREDO).update(req.body).digest("hex")
  const recebido = req.get("X-Arca-Signature") ?? ""
  const ok = recebido.length === esperado.length &&
    crypto.timingSafeEqual(Buffer.from(recebido), Buffer.from(esperado))
  if (!ok) return res.sendStatus(401)
  const evento = JSON.parse(req.body)
  console.log(evento.event, evento.detail)
  res.sendStatus(204)
})
```

Para trocar o segredo sem apagar o webhook, use **Gerar novo segredo** (na API, `PATCH /api/webhooks/{id}` com `"rotate_secret": true`). Pausar um webhook mantém a configuração e só suspende as entregas.

7.4 Prometheus

O gateway expõe `GET /metrics` no formato de exposição de texto do Prometheus. A rota exige `Authorization: Bearer` com uma sessão ou, de preferência, uma chave de acesso com papel Leitura.

prometheus.yml

```
scrape_configs:
  - job_name: arca
    scrape_interval: 15s
    metrics_path: /metrics
    authorization:
      type: Bearer
      credentials: arca_3f9c...        # chave de acesso com papel Leitura
    static_configs:
      - targets: ["localhost:8080"]
```

Métrica	O que mede
`arca_up`	1 quando o gateway alcança o servidor
`arca_ops_per_second` · `arca_commands_processed_total`	Vazão e total de comandos
`arca_connected_clients` · `arca_uptime_seconds`	Conexões abertas e tempo no ar
`arca_memtable_bytes` · `arca_immutable_memtable_bytes` · `arca_wal_bytes`	Memória de escrita e tamanho do log
`arca_sst_files{level}` · `arca_sst_bytes{level}`	Arquivos e bytes de cada nível, L0 a L6

Copyright (©) 2026 Erik Pimenta

Métrica	O que mede
`arca_compactions_total` · `arca_compaction_bytes_read_total` · `arca_compaction_bytes_written_total`	Trabalho da compactação
`arca_write_amplification` · `arca_block_cache_hit_ratio`	Amplificação de escrita e acerto do cache de blocos
`arca_keyspace_hits_total` · `arca_keyspace_misses_total` · `arca_expired_keys_total`	Acertos, faltas e chaves expiradas
`arca_net_input_bytes_total` · `arca_net_output_bytes_total`	Tráfego de rede do servidor
`arca_estimated_keys`	Estimativa de chaves e linhas
`arca_gateway_http_requests_total{code}` · `arca_gateway_sse_clients`	Requisições do gateway por status HTTP e painéis conectados

Exemplos de consulta (PromQL)

```
rate(arca_commands_processed_total[1m])          # comandos por segundo
sum(arca_sst_bytes)                               # bytes em disco
arca_sst_files{level="0"} >= 8                    # alerta: L0 acumulando (travamento de escrita)
```

Copyright (©) 2026 Erik Pimenta

08 Exportar e importar

Chaves e tabelas saem e entram em formatos simples, JSON e CSV, pelo painel ou pela API. Bom para levar dados de um ambiente a outro, popular um banco de teste ou alimentar uma planilha.

8.1 Chaves em JSON

Na tela **Chaves**, **Exportar** baixa as chaves que casam com o padrão da busca (até 100 000) num arquivo `arca-chaves-AAAAMMDD-HHMM.json`. **Importar** lê um arquivo no mesmo formato e pergunta o que fazer quando a chave já existe: sobrescrever ou pular. O TTL exportado é o tempo restante, em milissegundos.

```
{
  "format": "arca.keys",
  "version": 1,
  "exported_at": 1793110156760,
  "match": "sessao:*",
  "keys": [
    { "key": "sessao:42", "value": "ativo", "ttl_ms": 3412000 },
    { "key": "sessao:43", "value": "expirada", "ttl_ms": null }
  ]
}
```

```
curl -s "localhost:8080/api/keys/export?match=sessao:*" -H "Authorization: Bearer $TOKEN" -o
sessoes.json

jq '{keys: .keys, mode: "skip"}' sessoes.json | curl -s -X POST localhost:8080/api/keys/import
\
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' -d @-
# {"imported":2,"skipped":0,"failed":0}
```

Exportar exige papel Leitura; importar exige Operador. O corpo da importação pode ter até 16 MB.

8.2 Tabelas em CSV ou JSON

Na tela **Tabelas**, **Exportar** baixa todas as linhas da tabela em CSV (cabeçalho com os nomes das colunas, separador vírgula, aspas como no RFC 4180) ou em JSON (uma lista de objetos). **Importar CSV** lê um arquivo cuja primeira linha traz os nomes das colunas; colunas ausentes recebem o valor padrão ou `NULL`, e colunas `AUTOINCREMENT` podem ficar de fora.

Copyright (©) 2026 Erik Pimenta

```
curl -s "localhost:8080/api/tables/produtos/export?format=csv" -H "Authorization: Bearer
$TOKEN" -o produtos.csv

curl -s -X POST localhost:8080/api/tables/produtos/import -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d "$(jq -Rs '{csv: .}' produtos-novos.csv)"
# {"inserted":48,"failed":2,"errors":[{"line":17,"message":"chave primária duplicada"}]}
```

Cada linha é inserida com as mesmas regras do `INSERT`: uma linha com erro é contada em `failed` e as outras continuam. A resposta lista até 20 erros com o número da linha do arquivo. A API também aceita `{"rows": [{"coluna": valor}]}` no lugar de `csv`.

8.3 Boas práticas

- Crie um checkpoint antes de uma importação grande; se algo der errado, restaurar é apontar `dir` para ele.
- Para mover o banco inteiro entre máquinas, prefira copiar um checkpoint: é mais rápido e preserva tudo, inclusive índices e contadores.
- Os arquivos exportados podem conter dados sensíveis; guarde-os com o mesmo cuidado do banco.

Copyright (©) 2026 Erik Pimenta

09 API REST

O gateway expõe a API em `/api`. As respostas são JSON e as rotas, exceto login, saúde e o esquema OpenAPI, exigem o cabeçalho `Authorization: Bearer <token>`, com o token de uma sessão ou uma chave de acesso.

9.1 Rotas

Erros seguem sempre o formato `{"code": "...", "message": "...", "fields": {...}}`.

Rota	Papel mínimo	Descrição
`POST /api/auth/login`	—	Recebe `{email, password}` e devolve `{token, user, expires_at}`
`POST /api/auth/logout` · `GET /api/auth/me` · `PUT /api/auth/me/password`	Leitura	Sessão e conta
`GET` · `POST /api/tokens` · `DELETE /api/tokens/{id}`	Leitura	Chaves de acesso da própria conta (papel até o seu)
`GET /api/overview` · `GET /api/storage`	Leitura	Estatísticas do servidor, do motor e dos níveis
`GET /api/events?token=`	Leitura	SSE: evento `metrics` a cada segundo
`GET /api/integrations`	Leitura	Versão, endereço do servidor e URLs de OpenAPI e métricas
`GET /api/keys` · `GET /api/keys/value` · `GET /api/keys/export`	Leitura	Varredura com cursor, leitura e exportação em JSON
`PUT` · `DELETE /api/keys/value` · `POST /api/keys/expire` · `POST /api/keys/import`	Operador	Gravar, apagar, mudar TTL e importar
`GET /api/tables` · `GET /api/tables/{nome}/rows` · `GET /api/tables/{nome}/export`	Leitura	Tabelas, linhas com filtro, ordem e paginação, e exportação CSV/JSON
`POST` · `PUT` · `DELETE /api/tables/{nome}/rows` · `POST /api/tables/{nome}/import`	Operador	Inserir, alterar e apagar linhas pela chave primária; importar CSV
`POST /api/sql`	Leitura / Operador	Leitura só executa `SELECT`, `EXPLAIN`, `SHOW` e `DESCRIBE`
`GET` · `POST /api/queries` · `DELETE /api/queries/{id}`	Leitura	Consultas salvas (as suas e as compartilhadas)
`POST /api/storage/flush` · `compact` · `checkpoint` · `verify`	Operador	Operações do motor e verificação de integridade

Copyright (©) 2026 Erik Pimenta

Rota	Papel mínimo	Descrição
`GET /api/storage/checkpoints`	Leitura	Checkpoints com data, tamanho e arquivos
`DELETE /api/storage/checkpoints/{nome}`	Administrador	Apaga um checkpoint
`GET /api/clients` · `GET /api/slowlog`	Leitura	Conexões e comandos lentos
`POST /api/command` · `GET` / `PUT /api/config`	Administrador	Comando cru e configuração
`GET` · `POST` · `PATCH` · `DELETE /api/users` · `GET /api/audit`	Administrador	Usuários e auditoria
`GET /api/webhooks/events` · `GET` · `POST` · `PATCH` · `DELETE /api/webhooks`	Administrador	Webhooks e catálogo de eventos
`POST /api/webhooks/{id}/test` · `GET /api/webhooks/{id}/deliveries`	Administrador	Entrega de teste e últimas entregas
`GET /metrics`	Leitura	Métricas no formato do Prometheus (fora de `/api`)
`GET /api/health` · `GET /api/openapi.json`	—	Saúde e esquema OpenAPI 3.1

9.2 Exemplos

```
TOKEN=$(curl -s -X POST localhost:8080/api/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"operador@arca.dev","password":"Arca@2026"}' | jq -r .token)

curl -s -X POST localhost:8080/api/sql -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"query":"SELECT nome, preco FROM produtos ORDER BY preco DESC LIMIT 3"}'

curl -s -X PUT localhost:8080/api/keys/value -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"key":"config:tema","value":"escuro","ttl_s":600}'

curl -s -X POST localhost:8080/api/storage/verify -H "Authorization: Bearer $TOKEN"
# {"files":6,"bytes":3718204,"ok":6,"corrupt":[],"ms":41.7}
```

Copyright (©) 2026 Erik Pimenta

10 Configuração

O servidor lê, nesta ordem: o arquivo passado como primeiro argumento (formato `nome valor`, uma linha por parâmetro), as variáveis de ambiente `ARCA_*` e os argumentos `--nome valor`. Tamanhos aceitam `kb`, `mb` e `gb`.

10.1 Servidor

Parâmetro	Padrão	Variável	Em execução	Descrição
`bind`	127.0.0.1	`ARCA_BIND`	não	Endereço de escuta; `0.0.0.0` aceita conexões externas
`port`	7379	`ARCA_PORT`	não	Porta TCP
`dir`	dados	`ARCA_DIR`	não	Pasta do banco (checkpoints ficam em `checkpoints/` ao lado)
`requirepass`	vazio	`ARCA_PASSWORD`	sim	Senha do `AUTH`
`io-threads`	2	`ARCA_IO_THREADS`	não	Laços de eventos de rede
`worker-threads`	4	`ARCA_WORKER_THREADS`	não	Threads para comandos pesados
`appendfsync`	everysec	`ARCA_APPENDFSYNC`	sim	`always`, `everysec` ou `no`
`write-buffer-size`	4mb	`ARCA_WRITE_BUFFER_SIZE`	não	Tamanho da memtable
`max-file-size`	2mb	—	não	Tamanho alvo das SSTables de compactação
`level1-max-bytes`	10mb	—	não	Limite do L1 (cada nível seguinte ×10)
`block-cache-size`	64mb	`ARCA_BLOCK_CACHE_SIZE`	não	Cache de blocos
`bloom-bits-per-key`	10	—	não	Precisão do filtro de Bloom
`maxclients`	10000	—	sim	Conexões simultâneas
`timeout`	0	—	sim	Segundos ociosos até desconectar (0 desliga)
`proto-max-bulk-len`	64mb	—	não	Maior argumento aceito
`slowlog-log-slower-than`	10000	—	sim	Microssegundos para entrar no slowlog (−1 desliga)
`slowlog-max-len`	128	—	sim	Entradas guardadas no slowlog
`loglevel`	info	—	sim	`debug`, `info` ou `warning`

Copyright (©) 2026 Erik Pimenta

10.2 Gateway

Variável do gateway	Padrão	Descrição
`ARCA_ADDR`	127.0.0.1:7379	Endereço do servidor Arca
`ARCA_PASSWORD`	vazio	Senha do servidor
`ARCA_HTTP_ADDR`	:8080	Endereço HTTP do painel, da API e de `/metrics`
`ARCA_WEB_DIR`	../web/dist	Pasta do painel compilado
`ARCA_SEED`	true	Cria contas e dados de demonstração num banco sem usuários
`ARCA_SESSION_HOURS`	12	Validade das sessões do painel
`ARCA_POOL_SIZE`	16	Conexões RESP que o gateway mantém abertas com o servidor
`ARCA_LOG_LEVEL`	info	Nível de log do gateway

Chaves de acesso, webhooks e consultas salvas ficam guardados no próprio banco, em tabelas de sistema do painel; não há arquivo de configuração extra para eles.

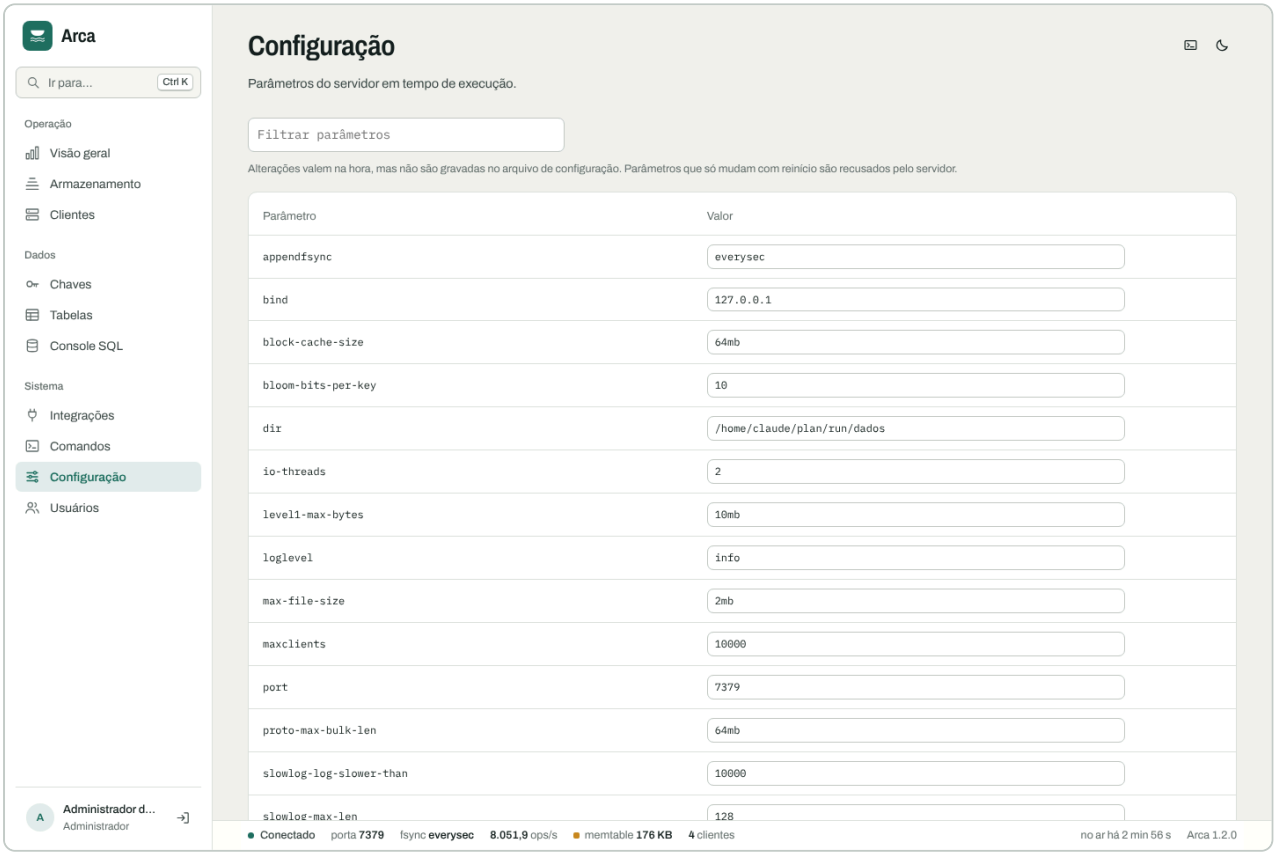


Figura 10.1 Configuração: parâmetros do servidor, com os que mudam em execução editáveis

11 Operação

Backup sem parar o servidor, verificação de integridade, monitoramento com Prometheus ou ferramentas de Redis, ajuste de desempenho e inspeção dos arquivos binários.

11.1 Backup com checkpoints

`ARCA.CHECKPOINT nome` (ou o botão **Criar checkpoint** no painel) grava a memtable e cria em `checkpoints/nome` uma cópia consistente do banco usando links físicos, sem parar o servidor. A pasta é um banco completo: para restaurar, pare o servidor e aponte `dir` para ela, ou copie seu conteúdo.

Na 1.2.0 a tela **Armazenamento** lista os checkpoints com data, tamanho e quantidade de arquivos, e um administrador pode apagar os antigos. Pela linha de comando, use `ARCA.CHECKPOINTS`.

```
arca-cli -a senha ARCA.CHECKPOINT antes-da-migracao
arca-cli -a senha ARCA.CHECKPOINTS
# [{"name":"antes-da-migracao","created_ms":1793110156760,"bytes":3718204,"files":8}]
arca-cli -a senha ARCA.CHECKPOINTS DELETE antes-da-migracao
docker compose exec arca arca-cli ARCA.CHECKPOINT diario          # no Docker
docker run --rm -v persistent-key-value-server_dados:/d -v "$PWD":/b alpine \
  tar czf /b/backup.tgz -C /d checkpoints
```

Como os arquivos do checkpoint são links físicos para as mesmas SSTables, ele ocupa pouco espaço enquanto o banco não compacta esses arquivos. Depois que a compactação substitui as SSTables originais, o checkpoint passa a ser o único dono delas: apague os que não servem mais.

11.2 Verificação de integridade

`ARCA.VERIFY` (ou **Verificar integridade** na tela Armazenamento, papel Operador) lê todos os blocos de todas as SSTables da versão atual, confere o CRC32C de cada um e se as chaves estão em ordem, sem parar o servidor e sem poluir o cache de blocos. O resultado diz quantos arquivos e bytes foram lidos, quantos estão íntegros e quais falharam.

```
127.0.0.1:7379> ARCA.VERIFY
"{\"files\":6,\"bytes\":3718204,\"ok\":6,\"corrupt\":[],\"ms\":41.7}"
```

- Se algum arquivo aparecer em `corrupt`, pare de escrever, crie um checkpoint para preservar o estado e restaure o arquivo de um checkpoint anterior. O `arca-inspect sst` mostra o que ainda pode ser lido do arquivo danificado.

11.3 Monitoramento

- `/metrics` entrega as métricas no formato do Prometheus (capítulo 7), prontas para Grafana e alertas.

Copyright (©) 2026 Erik Pimenta

- `INFO` traz memória, persistência, estatísticas e armazenamento no formato do Redis, então ferramentas de monitoramento de Redis conseguem ler.
- `ARCA.STATS` devolve tudo em JSON, inclusive cada SSTable e as últimas compactações.
- Amplificação de escrita = (bytes do usuário + flushes + compactação) ÷ bytes do usuário. Valores entre 2 e 10 são normais.
- Travamentos de escrita crescem quando o L0 acumula arquivos mais rápido do que a compactação consegue descer.

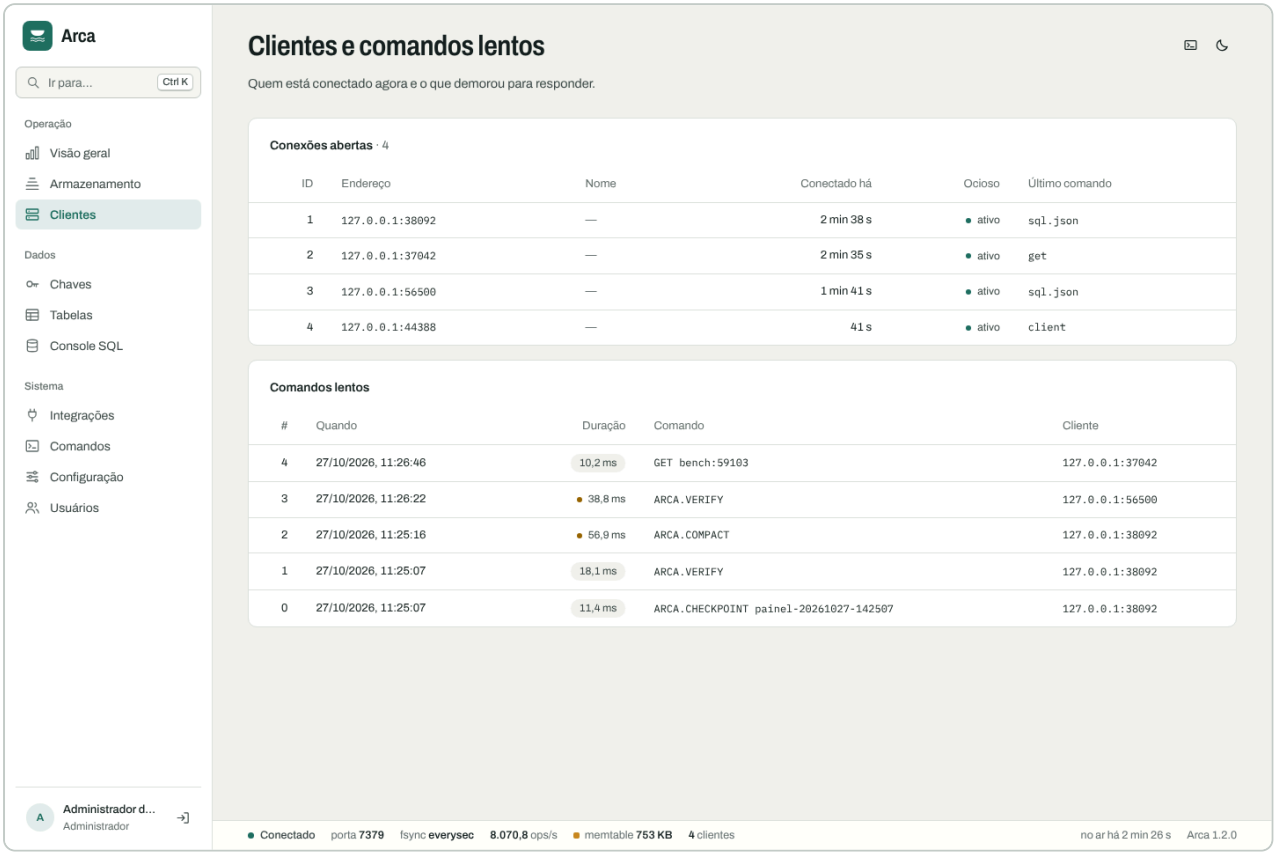


Figura 11.1 Clientes conectados e comandos lentos

11.4 Ajuste de desempenho

Sintoma	Ajuste
Muitos travamentos de escrita	Aumente `write-buffer-size` (menos flushes) e `level1-max-bytes`
Leituras lentas em dados frios	Aumente `block-cache-size`; `bloom-bits-per-key 14` reduz leituras inúteis
Latência alta com muitos clientes	Aumente `io-threads` até o número de núcleos
SQL pesado atrasando outros comandos	Aumente `worker-threads` e crie índices para os filtros mais usados
Precisa de durabilidade total	`appendfsync always` (cada escrita espera o fsync; o group commit reduz o custo)

Copyright (©) 2026 Erik Pimenta

11.5 Medindo

```
$ arca-bench -c 4 -n 60000 -P 32 -d 100
SET   176132 ops/s   p50   6.2 us   p99 210.7 us   erros 0
GET   327463 ops/s   p50   8.7 us   p99  65.3 us   erros 0
```

Valores medidos num contêiner Linux de 2 núcleos com `appendfsync everysec`; o resultado depende do disco e da CPU.

11.6 Inspecionando os arquivos

```
arca-inspect check dados                      # confere todas as SSTables (servidor parado)
arca-inspect sst dados/000042.sst --tudo      # propriedades e cada entrada
arca-inspect wal dados/000043.wal             # lotes do log
arca-inspect manifest dados/MANIFEST-000044   # histórico de versões
```

Com o servidor no ar, prefira `ARCA.VERIFY`: ele confere só os arquivos da versão atual e não disputa a trava do diretório.

Copyright (©) 2026 Erik Pimenta

12 Estendendo a Arca

Comandos novos, funções SQL, o motor embutido em outro programa e telas novas no painel.

12.1 Um comando novo

Os comandos ficam em `src/server/commands.cpp`. Escreva a função na struct `Commands` e registre o nome na tabela `CommandTable()` com a aridade (positiva é exata, negativa é mínima) e as flags.

```
static void Dobro(Server& s, Connection& c, Args& a, std::string* out) {
    KvRecord rec;
    if (!CheckRead(s, ReadKv(s, c, a[1], &rec), out)) return;
    resp::AppendBulk(out, rec.value + rec.value);
}

{"DOBRO", 2, kRead, Commands::Dobro},
```

Use `kWrite` para comandos que gravam, `kHeavy` para mandar ao pool de workers e `kNoMulti` para impedir o uso dentro de `MULTI`. Os comandos da 1.2.0 (`GETEX`, `COPY`, `ARCA.VERIFY`...) são bons exemplos de leitura e escrita na mesma operação.

12.2 Uma função SQL

Inclua o nome em `ValidateFunctions` e o cálculo no ramo `kFunction` de `Eval`, em `src/sql/engine.cpp`.

12.3 Embutindo o motor

A biblioteca `arca_engine` não depende da rede nem do SQL. Inclua `arca/db.hpp`, ligue com `arca_engine` no CMake e use `DB::Open`, `Put`, `Get`, `Delete`, `Write` com lotes, `NewIterator`, `GetSnapshot`, `Flush`, `CompactAll`, `Checkpoint` e `GetStats`. O exemplo completo está em `examples/embedded.cpp`.

12.4 Gateway e painel

Rotas novas entram em `gateway/internal/api/`; o cliente RESP com pool em `gateway/internal/resp` já cuida de autenticação e reconexão. Para que uma ação nova dispare webhooks, registre-a na auditoria com um nome de evento do catálogo. Telas novas entram em `web/src/pages/` e na lista de páginas de `web/src/App.tsx`, que também alimenta a paleta de comandos e os atalhos. Para manter a identidade visual, use os componentes de `web/src/components/ui.tsx` e os tokens do topo de `web/src/styles.css`; estilos das funcionalidades da 1.2.0 ficam em `web/src/features.css`.

Copyright (©) 2026 Erik Pimenta

13 Solução de problemas

Os problemas mais comuns, a causa provável e como resolver.

13.1 Problemas comuns

Problema	Causa provável	Solução
o diretório já está em uso por outro processo	Outro `arca-server` abriu a mesma pasta	Pare o outro processo ou use outro `dir`
bind na porta 7379 falhou	A porta está ocupada	Use `--port` ou pare o outro serviço
NOAUTH autenticação necessária	O servidor tem `requirepass`	Use `AUTH senha` ou `-a senha`
Painel mostra "Arca fora do ar"	O gateway não alcança o servidor	Confira `ARCA_ADDR`, a senha e `docker compose logs arca`
N arquivo(s) SSTable ausentes na abertura	Arquivo apagado ou cópia incompleta	Restaure de um checkpoint; o MANIFEST aponta exatamente o que falta
descartando N bytes no log da abertura	Queda no meio de uma gravação	Normal: só a cauda incompleta é descartada
`ARCA.VERIFY` lista arquivos em `corrupt`	Disco com setor ruim ou arquivo alterado por fora	Crie um checkpoint, troque o disco se preciso e restaure o arquivo de um checkpoint anterior
Muitos "travamentos de escrita"	Compactação mais lenta que a entrada	Veja o ajuste de desempenho no capítulo 11
chave primária duplicada	`INSERT` com chave que já existe	Use `UPDATE` ou outra chave
Importação com linhas em `failed`	Linhas que violam as regras da tabela	Corrija as linhas indicadas em `errors` (número da linha do arquivo) e importe só elas
Chave de acesso devolve 401	Chave revogada, vencida ou copiada pela metade	Crie outra em Integrações; a chave completa só aparece na criação
Chave de acesso devolve 403	O papel da chave não permite a rota	Crie uma chave com o papel necessário (até o seu)
Prometheus mostra o alvo como *down* com 401	`/metrics` exige Bearer	Configure `authorization` no `scrape_config` com uma chave de papel Leitura
Webhook com entregas falhando	URL inacessível, resposta fora de 2xx ou demora acima de 5 s	Veja o status e o erro das últimas entregas e use **Enviar teste** depois de corrigir
Assinatura do webhook não confere	Corpo alterado antes da verificação ou segredo trocado	Calcule o HMAC sobre o corpo cru, antes de interpretar o JSON, com o segredo atual
Conta do painel bloqueada	5 senhas erradas seguidas	Espere 5 minutos ou desbloqueie em Usuários
`iniciar.cmd` fecha sem subir	O Docker Desktop não está rodando	Abra o Docker Desktop e rode de novo

Copyright (©) 2026 Erik Pimenta